Prob 1. (25 points, 5 each) We can better understand the code when we can explain each part easily. Consider the single-line code below for checking if the pushbutton (a key) connected to Pin1 of GPIOA is pressed or not:

```
bool PA1_state = ((GPIOA->IDR & GPIO_PIN_1) == GPIO_PIN_1);
```

1. What is the value of `GPIO_PIN_1`?
2. If the key on PA1 is pressed and the value in IDR corresponding to Pin 1 is 1, what is the value of `GPIOA->IDR & GPIO_PIN_1`?
3. (Continue Part 2.) What is the value of `(GPIOA->IDR & GPIO_PIN_1)== GPIO_PIN_1`?
4. If the key on PA1 is released and the value in IDR corresponding to Pin 1 is 0, what is the value of `GPIOA->IDR & GPIO_PIN_1`?
5. (Continue Part 4.) What is the value of `(GPIOA->IDR & GPIO_PIN_1)== GPIO_PIN_1`?

Solution:

1. (5 points) What is the value of `GPIO_PIN_1`? **0x0000_0002**
2. (5 points) If the key on PA1 is pressed and the value in IDR corresponding to Pin 1 is 1, what is the value of `GPIOA->IDR & GPIO_PIN_1`? **0x0000_0002**
3. (5 points) Continue the above question. What is the value of `(GPIOA->IDR & GPIO_PIN_1)== GPIO_PIN_1`? **True**
4. (5 points) If the key on PA1 is released and the value in IDR corresponding to Pin 1 is 0, what is the value of `GPIOA->IDR & GPIO_PIN_1`? **0x0000_0000**
5. (5 points) Continue the above question. What is the value of `(GPIOA->IDR & GPIO_PIN_1)== GPIO_PIN_1`? **False**

-:-

Remarks about the solution:

1. `GPIO_PIN_1` is a mask for Pin1, which should have a 1 at bit 1, so it's least 4 significant bits should be `0b0010` => it hexadecimal value should be **0x0000_0002**.
2. If the key on PA1 is pressed and the value in IDR corresponding to Pin 1 is 1, the value of `GPIOA->IDR & GPIO_PIN_1` should be **0x0000_0002** as both number has a 1 at bit 1. The values of other bits will be zeroed by the 0s of `GPIO_PIN_1`.
3. Straightforward.
4. Bit 1 of IDR corresponding is 0, hence `GPIOA->IDR & GPIO_PIN_1` = **0x0000_0000**.
5. Straightforward.

-:-

Prob 2. (15 points.) Consider the problem of estimating the response time for an interrupt. Suppose we need to go the following operations to handle the interrupt with assumptions below:

- It takes the MCU 8 clock cycles for stacking 8 registers. In the meantime, the address of the ISR is fetched.

- We run 10 instructions in the ISR on the MCU which has a 3-stage pipeline. This running is done immediately after the stacking.
- Immediately after the running of the ISR, the MCU uses 8 more cycles to unstack the above registers. Now, we go back the main program.

What is total number of clock cycles that the main program is *interrupted* (suspended) during which the processor executes all the above operations?

Solution:

- To run the 10 instructions in ISR, we need to use 3 + (10-1) = 12 clock cycles.
- The stacking and unstacking tasks 16 clock cycles.
- The total time is 28 clock cycles.

> -:-

Remarks about the solution:

- One key issue is to figure out how many clock cycles we need to execute the 10 instructions.
- Another key issue to understand that we need to add all the time consumed by the stacking / unstacking and the execution of the ISR.

> -:-

Prob 3. (5 points.) Suppose we read the exception number from PSR as 22. What is the IRQn number of the interrupt?

Solution:

IRQn = PSR number - 16 = 6.

> -:-

Remarks about the solution:

- This is just a simple application of the equation in Section 8.4.3 in Lctr 8.

> -:-

Prob 4. (10 points.) Consider the definition of pointers in Section 8.6.1 of Lctr 8. What are the addresses of the structs for NVIC and SysTick timer, respectively?

Solution:

- Address for the struct for NVIC is 0xE0000_E100.
- Address of the struct for the SysTick timer is 0xE0000_E010.

> -:-

Remarks about the solution:

- These are the same as the other address-definition problems. We just need to add the numbers for a certain address together.

-:-

Prob 5. (10 points.) Consider the definition of the NVIC struct of Lctr 8. Assume the address of the NVIC struct is at `0xE000_0000`.

- What is the address of ICER[4] (element 4 of array ICER)?
- What is the address of IP[4]?

Solution:

- The address of ICER[4] is at `0xE000_0000` + `0x90`. The reason is that each element of ICER has 32 bits.
- The address of IP[4] is at `0xE000_0000` + `0x304`. The reason is that each element of IP has 8 bits.

-:-

Prob 6. (10 points.) Consider a certain interrupt which has an IRQn being 69. Assume we need to clear its pending bit.

- Determine which of the ICPRs we need to write to clear its pending bit? (For example ICPR[3])
- What is the location of the bit to write? (For example, bit 22.)

Solution:

- 69 » 5 = 2, so we need to write to ICPR[2].
- 69 % 32 = 5, so we need to on the location corresponding to bit 5.

-:-

Remarks about the solution:

- These problems are addressed in Section 8.6.2 of Lctr 8.

-:-

Prob 7. (35 points total, 5 points for the first 3 each and 10 points for the last 2 each) We have learned in Lctr 9 about how to set up the interrupt priority (IP) register using the `NVIC_SetPriority` function. Here we put that knowledge into use. Assume:

- We only use the most significant 4 bits of the byte in each IP register.
- We set up the system to use TWO bits for preemption priority.

Now, we consider the following code to set up the priorities of three interrupt sources, represented by IRQn1, IRQn2, and IRQn3, respectively.

```
1       (1)  NVIC_SetPriority(IRQn1, 12);
2       (2)  NVIC_SetPriority(IRQn2, 7);
3       (3)  NVIC_SetPriority(IRQn3, 3);
```

Answer the following questions:

1. What is the value of `NVIC->IP[IRQn2]` in binary ?
2. Can IRQn1 preempt IRQn2? (Give your reasons.)
3. Can IRQn3 preempt IRQn2? (Give your reasons.)
4. If we **want** a new IRQn4 to be able to preempt IRQn2, what will be the range of priority number we can use when setting it up?
5. If we **do not want** the new IRQn4 to be able to preempt IRQn2, what will be the range of priority number we can use when setting it up?

Solution:

1. The value of IP[IRQn2] is 0b01_11_0000 in binary. (The reason to separate the most significant 4 bits is to stress the preemption priority group.)
2. IRQn1 has a higher priority number, which means a lower priority, than IRQn2. Hence, IRQn1 cannot preempt IRQn2.
3. IRQn3 has a lower priority number, which means a higher priority, than IRQn3. Now, let's take a look at the preemption priority number. IP[IRQn3] = 0b00_11_0000; so, its preemption priority number is 0, lower than 1, the preemption priority number of IRQn2. Hence, it can preempt IRQn2.
4. If we want IRQn4 to be able to preempt IRQn2, its preemption priority number should be 0, lower than the preemption priority number of IRQn2. Its sub-priority number can be anything. So, the range of the priority number should be from 0 to 3.
5. If we do not want IRQn4 to be able to preempt IRQn2, its preemption priority number **should be no less than** 1, the value for preemption priority number of IRQn2. The sub-priority number can be anything. So, the range of the priority number should be from `0b01_00` to `0b11_11` or 4 to 15.

> -:-

Remarks about the solution:

- Because we only use the most significant 4 bits of the 8-bit register, we need to put the value 7 to the most significant 4 bits, so that value should be `0b01_11_0000`.
- Parts 2 and 3 are straight forward.
- For Part 4, we know the preemption priority number already. As the sub-priority number can be anything and we have two bits for this field, the range of the priority number should then be from 0 to 3.
- For Part 5, we know the preemption priority number should be from 1 to 3, and the sub-priority number can be anything. To find the range, we need to use the lowest values of both, `0b01_00`, to get the lower bound of the range and the highest values of both, `0b11_11`, to get the higher bound of the range.

> -:-

Prob 8. (20 points, 10 points each) Consider the code in Section 9.4.2 of the class notes of Lctr 9.

```
1      SYSCFG->EXTICR[0] &= ~MASK_FOR_EXTI0;     // Need to define this mask
2      SYSCFG->EXTICR[0] |= VALUE_FOR_EXTI0_PA; // Need to define this value
```

Assume we are using `EXTICR1` instead of `EXTICR[0]` and are given the info about EXTICR1 from the reference manual, shown in Figure 1.

9.2.3 SYSCFG external interrupt configuration register 1 (SYSCFG_EXTICR1)

Address offset: 0x08

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI3[3:0]				EXTI2[3:0]				EXTI1[3:0]				EXTI0[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Figure 1: EXTICR bit fields.

Also assume we want to configure **to use PC3 for EXTI3** instead of PA0 for EXTI0, which is controlled according to the pattern of multiplexer control as shown in Figure 2.

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:12 **EXTI3[3:0]**: EXTI 3 configuration bits
These bits are written by software to select the source input for the EXTI3 external interrupt.
0000: PA[3] pin
0001: PB[3] pin
0010: PC[3] pin
0011: PD[3] pin
0100: PE[3] pin
0101: PF[3] pin
0110: PG[3] pin
0111: PH[3] pin (only on STM32L496xx/4A6xx devices)
1000: PI[3] pin (only for STM32L496xx/4A6xx devices)

Figure 2: Control bits for EXTICR.

Now, the code should be changed to the following:

```
1       SYSCFG->EXTICR1 &= ~MASK_FOR_EXTI3;     // Need to define this mask
2       SYSCFG->EXTICR1 |= VALUE_FOR_EXTI3_PC; // Need to define this value
```

Determine the values of the following macros without checking the code:

1. `MASK_FOR_EXTI3`
2. `VALUE_FOR_EXTI3_PC`

Note that if you have difficulty deciphering the above given control, you can refer to Figure 6 of Lctr 7 and the related code.

Solution:

1. `MAST_FOR_EXTI3` is used to mask the bit field for EXTI3, which includes bits 12 to 15, according to Figure 1. Hence, it should be `0x0000_F000` (`= 15U << (3* 4)`).

2. `VALUE_FOR_EXTI3_PC` is used to configure the multiplexer to choose PC3, which should be `0b0010`, according to Figure 2. Hence, it should be `0x0000_2000` (`= 2U << (3* 4)`).

> -:-